

FruitNeRF++: A Generalized Multi-Fruit Counting Method Utilizing Contrastive Learning and Neural Radiance Fields

Lukas Meyer, Andrei-Timotei Ardelean, Tim Weyrich, and Marc Stamminger

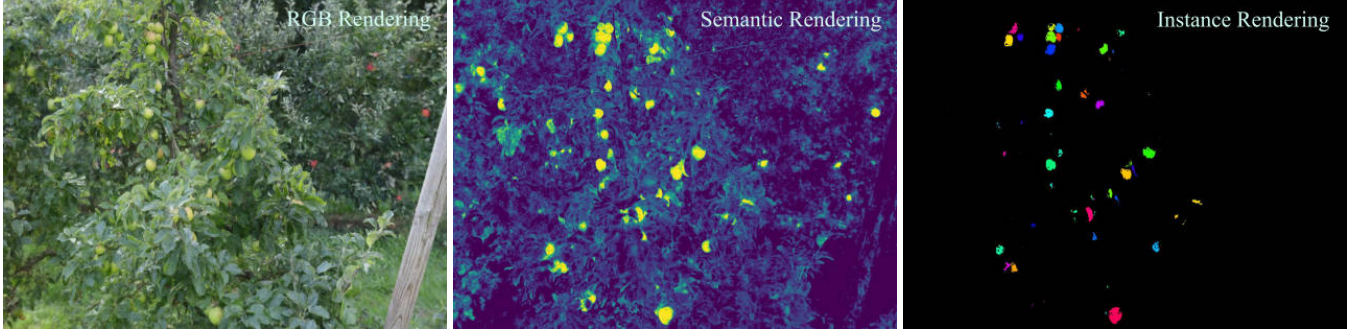


Fig. 1: Rendering of RGB, semantic and instance images. For visualization of the results visit the project page: meyerls.github.io/fruit_nerfpp.

Abstract—We introduce *FruitNeRF++*, a novel fruit-counting approach that combines contrastive learning with neural radiance fields to count fruits from unstructured input photographs of orchards. Our work is based on *FruitNeRF* [6], which employs a neural semantic field combined with a fruit-specific clustering approach. The requirement for adaptation for each fruit type limits the applicability of the method, and makes it difficult to use in practice. To lift this limitation, we design a shape-agnostic multi-fruit counting framework, that complements the RGB and semantic data with instance masks predicted by a vision foundation model. The masks are used to encode the identity of each fruit as instance embeddings into a neural instance field. By volumetrically sampling the neural fields, we extract a point cloud embedded with the instance features, which can be clustered in a fruit-agnostic manner to obtain the fruit count. We evaluate our approach using a synthetic dataset containing apples, plums, lemons, pears, peaches, and mangoes, as well as a real-world benchmark apple dataset. Our results demonstrate that *FruitNeRF++* is easier to control and compares favorably to other state-of-the-art methods.

I. INTRODUCTION

Counting fruits and estimating yield is a valuable tool for effective harvest and post-harvest management [1]. These tasks include planning the harvest workforce, packaging and wrapping, as well as storage and processing. A versatile multi-fruit counting framework is essential to ensure applicability across different types of orchards. Most existing frameworks are optimized for specific fruit types [2], [3], [4], [5], leaving

a gap in solutions that can handle diverse fruit varieties. To address this, we introduce a multi-fruit counting framework that unifies the detection process using a visual foundation model and generalizes object counting to accommodate varying shapes and appearances.

FruitNeRF [6] introduced a fruit-counting framework composed of two stages: one that is fruit-agnostic, and one that is fruit-specific. The generic component leverages a neural semantic field to integrate RGB and semantic modalities in a 3D space. The second component generates a semantic point cloud from the NeRF, focused exclusively on fruit data, and applies a cascaded clustering technique to accurately count the fruits. However, the cascaded point clustering method employed by *FruitNeRF* presents a key limitation: it relies on fruit-specific shape priors, restricting its applicability to generalized multi-fruit counting. To address this limitation, we propose *FruitNeRF++*, overcoming this issue by encoding instance-level information into a neural instance field using contrastive learning. This allows us to extract an instance-embedded point cloud from the neural field, which can be clustered in a shape-agnostic manner. Preliminary experiments demonstrate that this approach extends beyond fruit counting and is even applicable to more general object types.

The main contributions of our work are:

- We propose a novel multi-fruit, counting method that is shape-agnostic, by employing a neural instance field.
- We extend a multi fruit dataset¹ [6] with instance masks.
- We release the code of *FruitNeRF++*² upon acceptance.

II. RELATED WORK

A. Fruit Counting

With the rise of computer vision in agriculture, fruit counting in various orchard environments has garnered

*The authors are with the Visual Computing Erlangen (VCE), Friedrich-Alexander-Universität Erlangen-Nürnberg-Fürth, Germany, E-Mail: [lukas.meyer, timotei.ardelean, tim.weyrich, marc.stamminger]@fau.de

[†]This project is funded by the 5G innovation program of the German Federal Ministry for Digital and Transport under the funding code 165GU103B and the European Union's Horizon 2020 research and innovation program under the Marie Skłodowska-Curie grant agreement No 956585. The authors gratefully acknowledge the scientific support and HPC resources provided by the Erlangen National High Performance Computing Center (NHR@FAU) of the Friedrich-Alexander-Universität Erlangen-Nürnberg (FAU). The hardware is funded by the German Research Foundation (DFG).

¹Project website: meyerls.github.io/fruit_nerfpp

²*FruitNeRF++* code: github.com/meyerls/FruitNeRFpp

attention across multiple disciplines, including precision agriculture. Most research focuses on fruit types such as sweet peppers [7], strawberries [8], [9], apples [5], [3], [2], [11] and mangoes [4].

Liu et al. [4] propose a fruit counting system that processes image sequences. During the detection phase, a convolutional neural network is used to detect mangoes. In the tracking phase, a Kalman Filter, paired with an optical flow tracker, is applied to track individual fruits over time. The final count is achieved by triangulating the tracked fruit instances and localizing them in 3D space. Häni et al. [5] utilize a U-Net architecture [10] to predict segmentation masks for apples. These masks are projected into 3D to form a semantic point cloud. The fruit count is then obtained by clustering the point cloud and projecting the resulting clusters back onto the image plane. Similarly to Häni et al. [5] the approach of Gené-Mola et al. [2] uses instance segmentation masks. By clustering the semantic point cloud and projecting it back onto the image plane, they assign each 3D cluster a fruit id through overlap with the instance masks. *FruitNeRF* [6] is the first approach to work on different fruit types leveraging neural radiance fields and visual foundation models. By lifting the RGB and semantic images to 3D, a semantic point cloud is extracted. Clustering the semantic point cloud combined with fruit-specific templates leads to a fruit count.

B. Contrastive Learning

One of the prominent approaches for self-supervised learning of representations and dimensionality reduction is to employ a contrastive objective [12]. The essence of this class of methods is to use positive (similar) and negative (dissimilar) pairs of items as supervision. In general, these are significantly easier to obtain compared to exhaustive labels, and can sometimes be derived from simple priors, such as image augmentations when learning visual representations [13]. Thanks to the versatility of the concept, contrastive learning has been used in various contexts such as: unsupervised image classification [13], semantic segmentation [14], anomaly detection [15], anomaly clustering [16], or learning audio representations [17]. The various methods adopt different contrastive loss functions; some of the most common include the hard margin contrastive loss [12], triplet loss [18], ArcFace [19], and NT-Xent [20].

Our method adapts the InfoNCE loss [21] to leverage multiple positive and negative pairs, which are mined from 2D instance segmentation masks. The contrastive objective guides the training of our neural instance field through volumetric rendering, yielding a 3D-consistent instance field. Most related to our work is the use of contrastive learning for optimizing a 3D representation [22], discussed in the following subsection.

C. Neural Panoptic Fields

Since the introduction of neural radiance fields (NeRFs) [23] numerous advances have been made mainly focusing on the improvement of quality [25], inference [26], [27] and training time [28], [29]. Additionally NeRFs have been extended by both a semantic [30] and instance [31]

field component. *SemanticNeRF* [30] extends the default architecture by a semantic field to jointly optimize for semantics with rgb and density. Thus, they obtain a multi-view-consistent rendering of the assigned semantic classes. *InstanceNeRF* [31] uses instance masks that are already view-consistent; this enables the instance id of every object to be learned directly through an underlying instance field.

Panoptic Lifting [32] and Contrastive Lift [22] address a similar problem to ours, in that the methods seek to reconstruct a 3D panoptic/instance field from 2D masks. In Panoptic Lifting, the predicted 2D instances are not 3D-consistent; to solve this inconsistency, the method computes injective mappings from 2D instances to 3D surrogate identifiers by solving linear assignment problems. However, the mappings have to be computed repeatedly, imposing a large computational cost. Contrastive Lift avoids an explicit mapping by employing a contrastive objective to learn a field of instance embeddings. Here, the authors render the feature embeddings and utilize the 2D instance masks to coalesce similar and penalize dissimilar features. To improve the robustness of the method, the authors introduce an additional *concentration* loss and use a *slow* teacher for training the instance field. We also use contrastive learning to optimize a field of instance embeddings; however, we use a different loss function that avoids the overhead of maintaining two different (teacher and student) networks. Moreover, our approach is tailored for small objects, which are generally difficult to model. This is done by adjusting the pixel sampling (Sec. IV-C.3) to focus on the pairs that are hard to distinguish.

III. PRELIMINARIES

As our method builds on *FruitNeRF* [6], we summarize here the relevant components. *FruitNeRF* consists of a 3D representation that models the geometry and semantics, and an elaborate clustering algorithm that identifies individual fruits. To represent the geometry, a NeRF [23] is used, which is a neural representation that implicitly encodes a scene through a continuous function $\mathcal{F}$. Given a set of images and their camera parameters, a neural network is optimized to learn geometric and appearance properties of the scene. This is achieved by optimizing a density field $\mathcal{F}_\sigma: \mathbf{x} \rightarrow \sigma$ that maps a spatial coordinate $\mathbf{x} \in \mathbb{R}^3$ to a volume density σ . To obtain the RGB radiance $\mathbf{c}$, the appearance field $\mathcal{F}_c: (\mathbf{x}, \mathbf{d}) \rightarrow \mathbf{c}$ takes the position $\mathbf{x}$ and view direction $\mathbf{d}$ as input. The composed color of a pixel is determined by marching along a camera ray $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$ that originates in the camera center $\mathbf{o}$. The estimated color $\hat{\mathbf{C}}$ is obtained through volumetric rendering by querying $\mathcal{F}$ in K points along the ray $\mathbf{r}$ and accumulating the color values based on the density:

$$\hat{\mathbf{C}}(\mathbf{r}) = \sum_{k=1}^K \hat{T}(t_k) \alpha(\sigma(t_k) \delta_k) \mathbf{c}(t_k), \quad (1)$$

$$\text{where } \hat{T}(t_k) = \exp\left(-\sum_{a=1}^{k-1} \sigma(t_a) \delta_a\right).$$

Here $\delta_k = t_{k+1} - t_k$ defines the distance between two adjacent points and the transmittance probability is given by $\alpha(x) =$

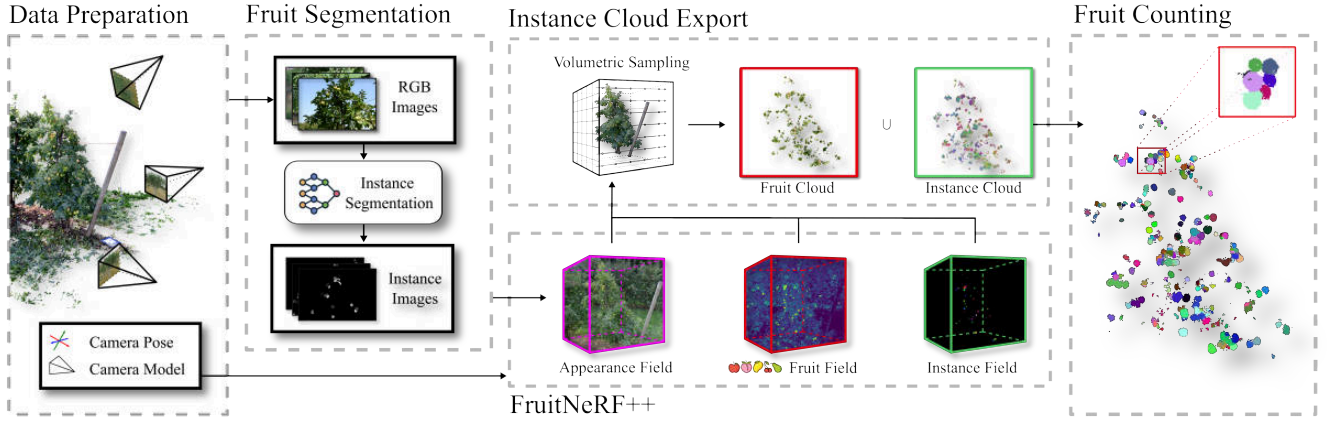


Fig. 2: Pipeline of *FruitNeRF++*. For the images we recover both intrinsic and extrinsic camera parameters. We then extract semantic and instance masks for arbitrary fruit types using SAM [39] and Detic [41]. The data are used to train a neural radiance field with a neural appearance, semantic (fruit) and instance field. By clustering the combination of a fruit and instance point cloud we obtain a precise fruit count.

$1 - \exp(-x)$. An illustrative RGB rendering of a fruit tree is visualized in Fig. 1 on the left.

Additionally, *FruitNeRF* [6] extends the NeRF approach by a neural semantic field [30] to lift 2D semantic information into 3D. Along with the color, a semantic field $\mathcal{F}_s: \mathbf{x} \rightarrow s \in \mathbb{R}$ is introduced, which maps a spatial coordinate to a semantic value. The estimation of the semantic $\hat{S}$ for a given pixel is computed similarly to $\hat{C}(\mathbf{r})$, but instead of colors $c(t_k)$, semantics $s(t_k)$ are accumulated according to

$$\hat{S}(\mathbf{r}) = \sum_{k=1}^K \hat{T}(t_k) \alpha(\sigma(t_k) \delta_k) s(t_k) . \quad (2)$$

The semantics rendering is depicted in Fig. 1 in the middle. By volumetric sampling of the implicit neural fields a point cloud is obtained that contains semantic points with a high density. Such semantic point cloud is referred to as a fruit point cloud. Afterwards, a cascaded clustering approach is applied in *FruitNeRF*, isolating each fruit to obtain a fruit count. *FruitNeRF* uses a fruit-specific template matching approach to subdivide clusters. The limitation of the clustering approach is that it needs fruit-templates for different type of fruit and varying template sizes for different grow stages.

To avoid this need for fruit-specific templates, we introduce with *FruitNeRF++* the concept of neural instance fields to not only learn semantics but also an instance identifier for each fruit in the scene. This allows us to combine semantic and instance information in the clustering stage and forego the template matching.

IV. FRUITNERF++

In this chapter we introduce the individual components of the *FruitNeRF++* pipeline, which is visualized in Fig. 2.

A. Data Preparation

Both our synthetic and real-world datasets consist of sets of unordered RGB images. The synthetic dataset incorporates the models from *FruitNeRF*, consisting of different fruit trees [33]: apple, plum, lemon, pear, peach, and mango. For each

tree model, we render 300 images from the upper hemisphere and extract the extrinsic and intrinsic camera parameters using the BlenderNeRF plugin [34]. In addition to rendering color and semantic images, we extend the dataset by additionally rendering the ground-truth instance masks of the fruits. A visualization of the dataset is shown on the project website.

The FUJI [35] dataset consists of 582 images taken from 12 trees at a commercial apple orchard. To recover the camera poses, we applied COLMAP [36] to each side and manually registered both sides. The semantic and instance masks were generated using the visual foundation models *Grounded-SAM* [37], [38], [39] and Detic [41]. All images for training were downsampled to a resolution of $1296 \text{ px} \times 864 \text{ px}$.

B. Fruit Segmentation

To achieve a unified fruit counting approach we utilize *Grounded-SAM* [37] and Detic [41] to predict precise instance segmentation masks without training a custom model. Both foundation models have the capability to generalize well to different types of fruit without the need for fine-tuning.

Grounded-SAM uses the name of each fruit as a text prompt, except for the fruit mango and plum, where we added *apple* to the prompt due to poor masks with *mango* or *plum* only. *Grounded-SAM* [37] combines the open-set object detector Grounding DINO [38] with the open-world segmentation model SAM [39]. For SAM [39] we use the weights of SAM-HQ [40].

Grounding DINO generates precise bounding boxes for each image by leveraging textual information as an input condition. These bounding boxes are then used by SAM as box prompts to predict accurate instance masks.

As Detic [41] can detect every class available in the vocabulary collection LVIS (Large Vocabulary Instance Segmentation) [42] Detic [41] combines both the identification of bounding boxes and the assignment of categories to objects. For Detic we choose the default network configuration and selected LVIS (Large Vocabulary Instance Segmentation) [42] as a vocabulary collection. For each detected bounding

TABLE I: Segmentation evaluation of Grounded-SAM [37] and Detic [41] on synthetic data using Intersection over Union (IoU). For Grounded-SAM we used each fruit as a text prompt. For Detic we took an array of multiple fruit classes present in the LVIS [42] Vocabulary and applied it to all fruit images. Listed tags: apple, apricot orange_(fruit), peach, persimmon, mandarin_orange, pear, banana, mango, lemon, pumpkin, plum, grape, cherry, blackberry, fig_(fruit), blueberry, pinecone, raspberry, date_(fruit), almond, lime, clementine. The best performing segmentation model is highlighted in green.

IoU ($\uparrow$)	Grounded-SAM	Prompt	Detic	Classes
apple	0.659	apple	0.632	see caption
pear	0.519	pear	0.468	
plum	0.382	plum, apple	0.144	
lemon	0.556	lemon	0.514	
peach	0.662	peach	0.654	
mango	0.588	mango, apple	0.528	

box, Detic computes a CLIP [43] embedding vector and uses it to estimate a corresponding text label. By leveraging CLIP embeddings, Detic can predict object categories without having seen them during training. we select every fruit-related classes available in LVIS and filter the predictions in a post-processing step.

On average the masks predicted of *Grounded-SAM* and Detic over all synthetic data have an intersection over Union (IoU) of 0.561 and 0.49, respectively. A detailed overview for different fruits is listed in Tab. I. However, both approaches face the issue that when a fruit is partially occluded by an obstacle, such as a leaf or branch, it may be detected as two separate instances. This leads to the feature space of a fruit being partially separated and incorrectly counted as two separate instances in the clustering phase.

C. Neural Instance Field

In this section we describe the key components of *FruitNeRF++*: the neural instance field (Sec. IV-C.1), the proposed contrastive loss function (Sec. IV-C.2), the pixel sampler strategy (Sec. IV-C.3), and the training process (IV-C.4).

1) *Instance Rendering*: Similarly to Contrastive Lift [22], the aim of the instance field is to lift view-inconsistent 2D instance masks to 3D. Therefore, we optimize the 3D instance embeddings through contrastive learning using 2D instance masks predicted by a foundation visual model. Our approach makes important modifications targeted to fruit-counting, by improving the separation of small objects, which is described in the next section.

The instance field $\mathcal{F}_i: x \rightarrow i$ maps 3D coordinates x to feature embeddings $i \in \mathbb{R}^D$. The goal of the instance field is to map points that belong to the same fruit to similar embeddings, while keeping embeddings from different fruits separated. Thus the instance embedding acts as a fruit identifier. The rendering of the instance field uses the same approach as Bhalgat et al. [22]:

$$\hat{I}(\mathbf{r}) = \sum_{k=1}^K \hat{T}(t_k) \alpha(\sigma(t_k) \delta_k) i(t_k) . \quad (3)$$

An instance rendering after clustering (see more regarding our clustering approach in Sec. IV-E) is visualized in Fig. 1 on the right.

2) *Contrastive Loss Function*: We extend the base network by a semantic (also referred as Fruit) and instance field. An overview of the *FruitNeRF++* architecture is schematically illustrated in Fig. 3. For training the density and appearance field we use the original photometric loss proposed by Mildenhall et al. [23]. It computes the loss between the ground truth pixel's RGB value $C(\mathbf{r})$ and the predicted color value $\hat{C}(\mathbf{r})$ along each ray $\mathbf{r}$ as

$$\mathcal{L}_{\text{Photo}} = \frac{1}{|\mathcal{R}|} \sum_{\mathbf{r} \in \mathcal{R}} \|C(\mathbf{r}) - \hat{C}(\mathbf{r})\|_2^2 . \quad (4)$$

Here $\mathcal{R}$ is defined as the set of all rays. For optimizing the semantic field, the loss has to distinguish between the two classes: background and fruit pixels. Therefore, we employ a binary cross entropy loss:

$$\mathcal{L}_{\text{Sem}} = \frac{1}{|\mathcal{R}|} \sum_{\mathbf{r} \in \mathcal{R}} p(\mathbf{r}) \log \hat{p}(\mathbf{r}) + (1 - p(\mathbf{r})) \log(1 - \hat{p}(\mathbf{r})) . \quad (5)$$

To train the instance field, we adapt the InfoNCE loss from MoCo [24]:

$$\mathcal{L}_q = -\log \frac{\exp(\langle \mathbf{q}, \mathbf{q}_+ \rangle / \tau)}{\sum_{\mathbf{q}_- \in Q} \exp(\langle \mathbf{q}, \mathbf{q}_- \rangle / \tau) + \exp(\langle \mathbf{q}, \mathbf{q}_+ \rangle / \tau)} , \quad (6)$$

where τ represents the temperature of the softmax, $\mathbf{q}_+$ represents a positive pair with $\mathbf{q}$, and the denominator sums over all negative samples ($\mathbf{q}_-$) and one positive sample, $\mathbf{q}_+$.

In contrast, we have readily available multiple positive pairs for each sampled pixel, i.e., all pixels belonging to the same 2D instance. We find that a naive implementation, where each pair of pixels in one fruit is contrasted against

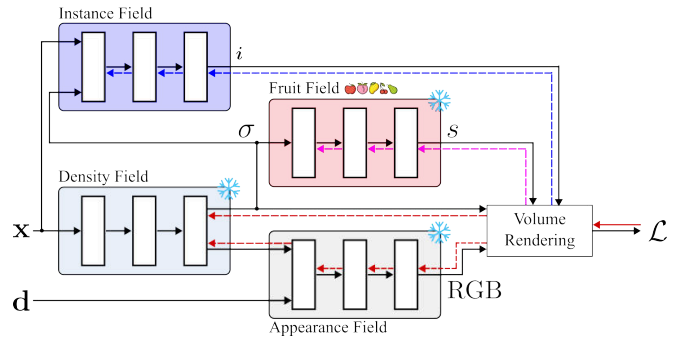


Fig. 3: Overview of the *FruitNeRF++* architecture, split up into four different components: density field, appearance field, fruit field, and instance field. The density field encodes the volume density σ , the appearance field the color RGB , the Fruit Field the semantic information about the fruit in space, and the instance field a feature vector i encoding information about the instance group of a point in space. The dashed arrow indicates the flow direction of the gradient. For training the different fields we employed a cascaded training scheme. First we train the density and RGB alone, followed by activating the semantic Fruit Field. Lastly, we freeze all three neural fields and train only the instance field. The figure is adapted from Özer et al. [44]. The colors of the dashed arrows correspond to their individual loss function in Eq. 8

all pixels from all other fruits, is suboptimal. Instead, we compute a fruit-prototype feature vector F_a , as the average of the features selected for that fruit: $F_a = \frac{1}{P} \sum_{\mathcal{P}(x_j)=a} \mathbf{i}(x_j)$, where $\mathcal{P}(x)$ is the fruit to which a point x belongs. For each pixel, we treat the fruit-prototype feature as the positive pair, and the prototypes of all other fruits as negative pairs. Thus, our contrastive loss is:

$$\mathcal{L}_{\text{contr}} = - \sum_{j=1}^{G \times L \times P} \log \frac{\exp(\langle \mathbf{i}(x_j), F_{\mathcal{P}(x_j)} \rangle / \tau)}{\sum_{a=1}^{G \times (L+1)} \exp(\langle \mathbf{i}(x_j), F_a \rangle / \tau)} . \quad (7)$$

where G , L , and P represent the number of groups, the number of fruits per group, and the number of pixels per fruit respectively; sampled as discussed in IV-C.3. The final training loss is finally given by:

$$\mathcal{L} = \mathcal{L}_{\text{Photo}} + \lambda_{\text{Sem}} \mathcal{L}_{\text{Sem}} + \lambda_{\text{contr}} \mathcal{L}_{\text{contr}} . \quad (8)$$

The weighting factors λ_{Sem} and λ_{contr} are set to 1, and we prevent both gradients to propagate through the density field.

3) *Pixel Sampling Strategies*: We observe that the uniform pixel sampling, customarily employed when training radiance fields, is inefficient in providing the training signal for the instance fields. That is because the large number of background pixels overshadows the small gradient updates needed to distinguish between different fruits. Therefore, we design specific pixel samplers for training different fields, tailoring the optimization process for the fruit-centric tasks.

The purpose of the RGB and semantics pixel sampler is to provide uniform coverage of all data to train the density, appearance (RGB) and semantic fields. Therefore, we keep here the default method used in Nerfstudio [45], which uniformly samples pixels across all available images.

The training of the instance field in *FruitNeRF++* is performed on a per-image basis, as the contrastive learning approach requires positive and negative pairs, which are only available within each image. The task of the instance pixel sampler is to form the pixel pairs for training, as visualized in Fig. 4. To obtain positive pairs, we use the 2D instance mask to select multiple points from a single fruit, which are optimized to have similar instance embeddings. By repeating the process above, we obtain groups of positive pairs. In turn, by matching pixels from different groups we obtain negative pairs which are optimized to be distinct through our contrastive loss (Eq. 7). Since it is harder to separate fruits that are (spatially) closer, we improve the training signal for such cases by mining local negatives (hard pairs). To obtain these pairs, for each fruit we select $L - 1$ nearest neighbours (in terms of distance between the 2D instance masks), and then sample P pixels per fruit. We generate G such groups of fruits, which provide hard (local) negatives (Fig. 4, right) and use all pairs formed from different groups as global negatives (Fig. 4 on the left).

4) *Cascaded Training*: The training of the semantic and instance fields is unstable if the representation of density in the neural field is poor. Therefore, we use a cascaded training method, by optimizing at first only the density and appearance fields. Then, the semantic field is added and trained jointly.

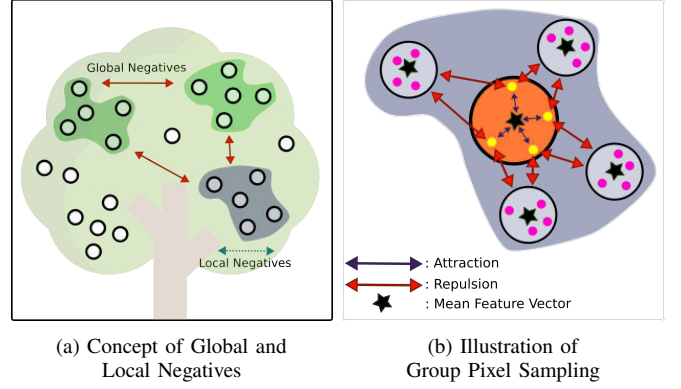


Fig. 4: In Fig. (a) we visualize the concept of local and global negatives. Local negatives are a collection of multiple fruits in near vicinity (see Sec. IV-C.3). By selecting these hard negatives, we enforce the features of neighbouring fruits to be distinct, facilitating their separation during the clustering stage. Global (weak) negatives are then used to separate distant fruits. Fig (b) visualizes both pixel sampling and local negatives in detail. The pixels from the orange center are denoted as positive and all others as negative. By computing the contrastive loss function between every pixel’s feature vector and the mean feature vector, we attract positive (yellow pixels) and repulse negative features (pink pixels).

Finally, we freeze the appearance and semantic fields and train only the instance field; the other parts of the network are frozen, as the instance pixel sampler may otherwise cause catastrophic forgetting.

D. Point Cloud Export

FruitNeRF++ gathers information about density, semantics, and instance segmentation within its different neural radiance fields. To export a point cloud after training, we effectively sample each field in the pipeline by uniformly sampling the space of our scene. This volumetric sampling is achieved by querying the network with every spatial point to extract relevant information.

The density field provides precise details about a point’s geometric significance, the appearance field captures its color composition, the fruit field indicates the presence of a fruit, and the instance field reveals the implicit affiliation to a specific fruit id. Since the density field contains information about the physical properties of the tree, and the semantic field identifies whether a point belongs to a fruit, we can combine both point clouds to obtain a fruit point cloud.

The instance point cloud, derived from sampling the instance field, includes a D -dimensional feature vector for each point, encoding the fruit instance. By combining the fruit point cloud with the instance point cloud, we create a point cloud that represents only the fruits, along with their corresponding D -dimensional feature vectors.

E. Fruit Counting via Multi-Modal Clustering

Our fruit-counting approach is radically different from the clustering concept used in *FruitNeRF* [6]. There, a cascaded clustering procedure is employed by separating first conglomerates of fruits from single fruits. Then, the conglomerates of fruits are subdivided and counted using

TABLE II: Fruit counting results on different fruit types with different models. The synthetic data have an image size of 1024 px $\times$ 1024 px and contain of 300 frames. Masks generated by *Grounded-SAM* [37] have the corresponding fruit type as input. Only for plum we set the text prompt to ‘apple’ & ‘plum’ and for mangoes to ‘apple’. As Detic [41] classifies every object in the image, we select all available fruit classes. The results refer to counting result and are obtained if the center of a fruit is close to the GT center. The best performing methods on each fruit is highlighted in green. *: results are taken from *FruitNeRF* [6].

<i>FruitNeRF</i> <i>GT Mask</i>					<i>FruitNeRF++</i> <i>GT Mask</i>			
Fruit Type	Count*	Precision* ($\uparrow$)	Recall* ($\uparrow$)	F1-Score* ($\uparrow$)	Count	Precision ($\uparrow$)	Recall ($\uparrow$)	F1-Score ($\uparrow$)
Apple	283/283	1	1	1	285/283	0.993	1	0.996
Plum	642/745	0.973	0.812	0.885	572/745	0.989	0.759	0.859
Lemon	321/326	0.993	0.963	0.978	312/326	1	0.957	0.978
Pear	237/250	1	0.944	0.971	252/250	0.992	1	0.996
Peach	148/152	1	0.973	0.987	153/152	0.993	1	0.996
Mango	929/1150	0.978	0.788	0.873	649/1150	0.988	0.557	0.713

<i>FruitNeRF++</i> <i>SAM Mask</i>					<i>FruitNeRF++</i> <i>Detic Mask</i>			
Fruit Type	Count	Precision ($\uparrow$)	Recall ($\uparrow$)	F1-Score ($\uparrow$)	Count	Precision ($\uparrow$)	Recall ($\uparrow$)	F1-Score ($\uparrow$)
Apple	283/283	0.996	0.996	0.996	284/283	0.996	1	0.998
Plum	254/745	1	0.341	0.509	137/745	1	0.184	0.311
Lemon	307/326	0.997	0.938	0.967	274/326	0.993	0.834	0.907
Pear	236/250	0.995	0.94	0.967	233/250	1	0.932	0.965
Peach	152/152	0.993	0.993	0.993	150/152	0.987	0.974	0.98
Mango	457/1150	0.987	0.392	0.561	324/1150	0.997	0.281	0.438

another clustering step, based on a fruit-specific template. However, a drawback of this method is that both stages (DBSCAN [46] and agglomerative clustering [47]) require several hyper-parameters, which heavily influence the results, making the approach impractical for a multi-fruit counting approach. In *FruitNeRF++*, we replace this procedure with a simpler clustering approach that is divided into spatial partitioning and multi-modal clustering.

1) *Spatial Partitioning*: Spatial partitioning uses *k*-Means [47] to divide the existing Euclidean fruit point cloud into *S* different partitions with similar point cloud size. The value of *S* is arbitrarily selected and simply aims to reduce the number of points in the multi-feature clustering step, enhancing computational efficiency.

2) *Multi-Modal Clustering*: The multi-feature clustering is applied to all *S* partitions independently. For each *S*, we cluster the point cloud using HDBSCAN [48] with a custom distance metric composed of an Euclidean and a cosine distance. That is, given two points $P_k = (i_k, x_k)$ and $P_l = (i_l, x_l)$, our metric computes the distance as follows:

$$d_{kl} = \lambda_c d_c(i_k, i_l) + \lambda_e d_e(x_k, x_l) \quad (9)$$

Here, $i \in \mathbb{R}^D$ represents the feature embedding, and x the 3D position. d_c is defined as the cosine distance: $d_c(a, b) = 1 - \frac{\langle a, b \rangle}{\|a\| \|b\|}$ which measures the distance between two feature vectors and is weighted by $\lambda_c = 1$. The Euclidean distance d_e between two points is weighted by $\lambda_e = 5$. A detailed evaluation of feasible λ_e is provided in Sec. V.

V. EVALUATION AND RESULTS

We evaluate *FruitNeRF++* on our synthetic dataset. We used the same parameters for all experiments with an instance embedding dimension of $D = 32$, a temperature of $\tau = 0.2$ and $\lambda_c = 1$ and $\lambda_e = 1$. In Tab. II the results show that our

approach outperforms *FruitNeRF* on most fruit types. Overall, *FruitNeRF++* with ground truth masks achieve an average F1-score of 0.925 compared to *Grounded-SAM*-generated masks at 0.832 and Detic masks at 0.776.

Secondly, we evaluate the impact of different feature embedding sizes D , which define the space that encodes the fruit identity. In this experiment we sweep from $D = 1$ to $D = 256$, set the temperature to $\tau = 0.2$ and the weighting parameters to $\lambda_e = 0$ and $\lambda_c = 1$. In Fig. 5 on the left the results for all three fruits are visualized. It can be seen that increasing the feature dimension leads to an increase in F1-Score as more fruit identities can be encoded. For apples, the F1-Score converges to 1 at $D = 16$, whereas for plums and mangoes the maximum is reached with a larger feature embedding size. The poor result for mangoes can be attributed to the large amount of fruits and high occlusions inside the tree.

Our next experiment looks at the contrastive loss function and its temperature parameter τ . A lower temperature increases the penalty on the hardest negatives, leading to more separation between local structures [49] and causes the loss to concentrate on points in its vicinity. In contrast, a higher (e.g. $\tau \rightarrow +\infty$) temperature reduces sensitivity to hard negatives and gives all negative points the same magnitude of penalties. A feasible τ aims for a reasonable distribution which is locally clustered and globally separated. From Fig. 5, center, it is evident that the feasible region is between 0.1 and 0.3 and peaks at 0.2. In the edge cases, the clustering is not able to properly separate the instance embeddings.

To investigate our custom clustering metric (Fig. 5, right), we fixed $\lambda_c = 1$ and varied λ_e from 0 to 10. The case $\lambda_e = \infty$ indicates $\lambda_c = 0$ and $\lambda_e = 1$. The clustering performance with only cosine distance is limited, yet increasing rapidly when combined with the Euclidean component. Removing the cosine component similarly results in a drop in performance.

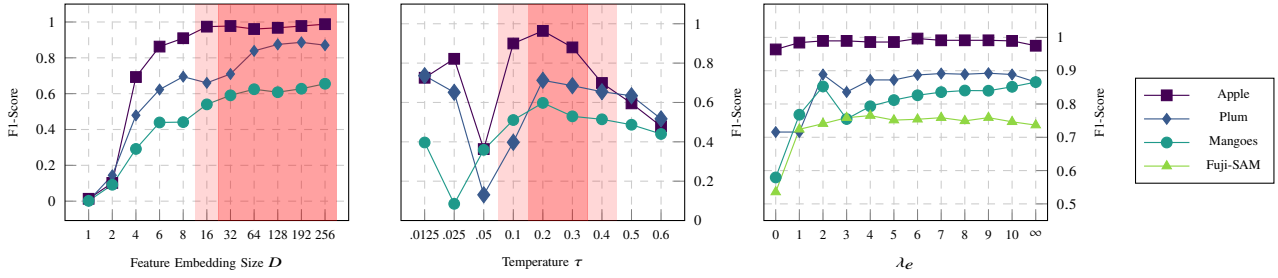


Fig. 5: Counting result on variation of the embedding size D is depicted on the left. Temperature is set to $\tau = 0.2$. $\lambda_e = 0$ and $\lambda_c = 1$ to show the impact of the embedding size only. In the middle the sweep over temperature τ is shown. Here we set $\lambda_e = 1$ and $\lambda_c = 1$. On the right shows the cluster performance via parameter sweep over λ_e while setting $\lambda_c = 1$. The red area describes the feasible area for parameters D and τ .

The minor contribution of feature embeddings to the clustering results (e.g. for apple) can be attributed to the fact that for synthetic data clustered fruits occur rarely and thus the main separation can be done in Euclidean space.

For the FUJI dataset, our method *FruitNeRF++* obtains an F1-Score of 0.765 using instance masks generated by *Grounded-SAM*, an embedding size of $D = 32$, temperature $\tau = 0.35$ and $\lambda_e = 5$. We sampled 600k points and chose $S = 40$ partitions. The results of our generalized multi-fruit approach compare well to the fruit-specific approach from Gené-Mola et al. [2] with an F1-Score of 0.881. The results of our counting algorithm compared to Gené-Mola et al. [2] can be attributed to the noisy poses and manual registration of both sides.

VI. LIMITATIONS

Our fruit counting method, *FruitNeRF++*, has limitations. Training on larger scenes, like the Fuji dataset, takes about 8 hours on an Nvidia A5000 (24GB VRAM), making it unsuitable for real-time use. The slow convergence of the instance field accounts for more than half of the training time. Additionally the results heavily depend on a good coverage of the tree, and noisy data significantly decreases the correctness of the implicit field and thus the counting result. This is mainly due to inaccurate poses, which cause the embeddings of one fruit to contaminate the neighbouring features. Lastly, the instance segmentation tends to predict two different instances if a leaf or branch visually separates a fruit; this causes the feature embeddings of the two halves to be pulled apart.

As we claim to provide a generalized fruit counting approach, challenges arise for fruits that grow in clusters, such as bananas, berries, or dates. Further research is needed to efficiently estimate cluster sizes and predict accurate cluster counts.

VII. CONCLUSION & OUTLOOK

We introduced *FruitNeRF++*, a novel method to count fruits from view-inconsistent instance segmentation masks using neural radiance fields. It encodes the instance identity of each fruit implicitly in the neural field and allows a simple clustering approach to obtain a fruit count. Compared to its predecessor *FruitNeRF*, our approach is fully agnostic to the type of fruit, regarding both the neural radiance field and the clustering. This also applies to any type of object as it does

not need any object shape priors. In an additional experiment with the Messy Rooms dataset [22] that contains 100 common household objects, we detected 99 objects. For a visualization visit our project website.

Recent work by Xu *et al.* [52] introduces a class-agnostic object counting approach that requires only a class name to estimate object counts automatically. However, as this method is currently designed for single 2D images, extending it to 3D or multi-view images could be a promising direction, potentially improving accuracy and robustness in fruit counting.

As a future direction for a NeRF-based counting approach, it is necessary to reduce the computational complexity. Using time-series images and incrementally building up the scene with SLAM [50], Gaussian Splatting [51] or PAGNeRF [7] could help achieve real-time performance. Additionally, temporal information could simplify fruit ID association and further reduce computation time.

ACKNOWLEDGEMENT

We extend our gratitude to **Adam Kalisz** for his unique Blender skills, **Jann-Ole Henningson** for proof reading our script, **Victoria Schmidt**, and **Annika Killer** for their invaluable assistance in evaluating the recorded apple trees.

REFERENCES

- [1] J. C. Miranda, J. Gené-Mola, M. Zude-Sasse, N. Tsoulas, A. Escolà, J. Arnó, J. R. Rosell-Polo, R. Sanz-Cortiella, J. A. Martínez-Casasnovas and E. Gregorio, "Fruit sizing using AI: A review of methods and challenges," *Postharvest Biology and Technology*, 2023.
- [2] J. Gené-Mola, R. Sanz-Cortiella, J. R. Rosell-Polo, J. R. Morros, J. Ruiz-Hidalgo, V. Vilaplana and E. Gregorio, "Fruit detection and 3D location using instance segmentation neural networks and structure-from-motion photogrammetry," *Computers and Electronics in Agriculture*, 2020.
- [3] X. Liu, S. W. Chen, S. Aditya, N. Sivakumar, S. Dcunha, C. Qu, C. J. Taylor, J. Das and V. Kumar, "Robust Fruit Counting: Combining Deep Learning, Tracking, and Structure from Motion," *arXiv*, 2018.
- [4] X. Liu et al., "Monocular Camera Based Fruit Counting and Mapping With Semantic Data Association," *IEEE Robotics and Automation Letters*, vol. 4, pp. 2296-2303, 2019.
- [5] N. Häni, P. Roy and V. Isler, "A comparative study of fruit detection and counting methods for yield mapping in apple orchards," *Journal of Field Robotics*, vol. 37, pp. 263-282, 2019.
- [6] L. Meyer, A. Gilson, U. Schmidt and M. Stamminger, "FruitNeRF: A Unified Neural Radiance Field based Fruit Counting Framework," *arXiv*, 2024.
- [7] S. Claus, M. Halstead, P. Zimmer, T. Laebe, E. Guclu, C. Stachniss, and C. McCool, "PAG-NeRF: Towards fast and efficient end-to-end panoptic 3D representations for agricultural robotics," *IEEE Robotics and Automation Letters*, 2023.

- [8] A. Riccardi, S. Kelly, E. Marks, F. Magistri, T. Guadagnino, J. Behley, M. Bennewitz, and C. Stachniss, "Fruit Tracking Over Time Using High-Precision Point Clouds," *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2023.
- [9] M. Sorour, P. J. From, K. Elgeneidy, S. Kanarachos, and M. Sallam, "Compact Strawberry Harvesting Tube Employing Laser Cutter," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2022.
- [10] O. Ronneberger, P. Fischer and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*, N. Navab, J. Hornegger, vol. 9351, pp. 234–241, 2015.
- [11] Y. Zhang, W. Zhang, J. Yu, L. He, J. Chen, and Y. He, "Complete and accurate holly fruit counting using YOLOX object detection," *Computers and Electronics in Agriculture*, vol. 198, p. 107062, 2022.
- [12] R. Hadsell, S. Chopra, and Y. LeCun, "Dimensionality Reduction by Learning an Invariant Mapping," *2006 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, 2006.
- [13] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A Simple Framework for Contrastive Learning of Visual Representations," *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020.
- [14] M. Hamilton, Z. Zhang, B. Hariharan, N. Snavely, and W. T. Freeman, "Unsupervised Semantic Segmentation by Distilling Feature Correspondences," *International Conference on Learning Representations*, 2022.
- [15] C. Guille-Escuret, P. Rodriguez, D. Vazquez, I. Mitliagkas, and J. Monteiro, "CADet: Fully Self-Supervised Out-Of-Distribution Detection With Contrastive Learning," *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [16] A.-T. Ardelean and T. Weyrich, "Blind Localization and Clustering of Anomalies in Textures," *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, 2024.
- [17] A. Saeed, D. Grangier, and N. Zeghidour, "Contrastive Learning of General-Purpose Audio Representations," *arXiv preprint*, 2020.
- [18] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A Unified Embedding for Face Recognition and Clustering," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015.
- [19] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, "ArcFace: Additive Angular Margin Loss for Deep Face Recognition," *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [20] K. Sohn, "Improved Deep Metric Learning with Multi-class N-pair Loss Objective," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 29, 2016.
- [21] A. van den Oord, Y. Li, and O. Vinyals, "Representation Learning with Contrastive Predictive Coding," *arXiv*, 2019.
- [22] Y. Bhalgat, I. Laina, J. F. Henriques, A. Zisserman and A. Vedaldi, "Contrastive Lift: 3D Object Instance Segmentation by Slow-Fast Contrastive Fusion," *Proceedings of the Thirty-seventh Conference on Neural Information Processing Systems*, 2023.
- [23] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi and R. Ng, "NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis," *Commun. ACM*, vol. 65, pp. 99–106, Jan. 2022.
- [24] Kaiming He, Haoqi Fan, Yuxin Wu, Saining Xie, and Ross Girshick, "Momentum contrast for unsupervised visual representation learning," *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [25] J. T. Barron, B. Mildenhall, D. Verbin, P. P. Srinivasan and P. Hedman, "Zip-NeRF: Anti-Aliased Grid-Based Neural Radiance Fields," *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [26] C. Reiser, S. Peng, Y. Liao and A. Geiger, "KiloNeRF: Speeding up Neural Radiance Fields with Thousands of Tiny MLPs," *CoRR*, vol. abs/2103.13744, 2021.
- [27] A. Yu, R. Li, M. Tancik, H. Li, R. Ng and A. Kanazawa, "PlenOctrees for Real-time Rendering of Neural Radiance Fields," *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [28] S. Fridovich-Keil, A. Yu, M. Tancik, Q. Chen, B. Recht and A. Kanazawa, "Plenoxels: Radiance Fields without Neural Networks," *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [29] T. Müller, A. Evans, C. Schied and A. Keller, "Instant Neural Graphics Primitives with a Multiresolution Hash Encoding," *ACM Transactions on Graphics (TOG)*, vol. 41, no. 4, pp. 102:1–102:15, July 2022.
- [30] S. Zhi, T. Laidlow, S. Leutenegger and A. J. Davison, "In-Place Scene Labelling and Understanding with Implicit Scene Representation," *CoRR*, 2021.
- [31] Y. Liu, B. Hu, J. Huang, Y.-W. Tai and C.-K. Tang, "Instance Neural Radiance Field," *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [32] Y. Siddiqui, L. Porzi, S. R. Bulò, N. Müller, M. Nießner, A. Dai and P. Kotschieder, "Panoptic Lifting for 3D Scene Understanding With Neural Fields," *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [33] XFrog Inc., LIBRARY: FRUIT TREES, 2020. [Online: <https://www.xfrog.com/product-page/library-fruit-trees>]
- [34] M. Raafat, "BlenderNeRF" (Version 5.0.0), 2023, [Computer software]. <https://doi.org/10.5281/zenodo.7926211>
- [35] J. Gené-Mola, R. Sanz-Cortiella, J. R. Rosell-Polo, J. R. Morros, J. Ruiz-Hidalgo, V. Vilaplana and E. Gregorio, "Fuji-SfM dataset: A collection of annotated images and point clouds for Fuji apple detection and location using structure-from-motion photogrammetry," *Data in Brief*, vol. 30, 2020.
- [36] J. L. Schönberger and J.-M. Frahm, "Structure-from-Motion Revisited," *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [37] T. Ren *et al.*, "Grounded SAM: Assembling Open-World Models for Diverse Visual Tasks," *arXiv*, 2024.
- [38] S. Liu *et al.*, "Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection," *arXiv*, 2024.
- [39] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. C. Berg, W.-Y. Lo, P. Dollár and R. Girshick, "Segment Anything," *arXiv*, 2023.
- [40] L. Ke, M. Ye, M. Danelljan, Y. Liu, Y.-W. Tai, C.-K. Tang, and F. Yu, "Segment Anything in High Quality," *Proceedings of the Neural Information Processing Systems (NeurIPS)*, 2023.
- [41] X. Zhou, R. Girdhar, A. Joulin, P. Krähenbühl and I. Misra, "Detecting Twenty-thousand Classes using Image-level Supervision," *arXiv*, 2022.
- [42] A. Gupta, P. Dollár and R. Girshick, "LVIS: A Dataset for Large Vocabulary Instance Segmentation," *arXiv*, 2019.
- [43] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, *International Conference on Machine Learning (ICML)*, 2021.
- [44] M. Özer, M. Weiherer, M. Hundhausen, and B. Egger, "Exploring Multimodal Neural Scene Representations With Applications on Thermal Imaging," *ArXiv*, 2024.
- [45] M. Tancik, E. Weber, E. Ng, R. Li, B. Yi, J. Kerr, T. Wang, A. Kristoffersen, J. Austin, K. Salahi, A. Ahuja, D. McAllister, and A. Kanazawa, "Nerfstudio: A Modular Framework for Neural Radiance Field Development," *ACM SIGGRAPH 2023 Conference*, 2023.
- [46] M. Ester, H.-P. Kriegel, J. Sander and X. Xu, "A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise," in *Knowledge Discovery and Data Mining*, 1996.
- [47] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot and É. Duchesnay, "Scikit-learn: Machine Learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
- [48] R. J. G. B. Campello, D. Moulavi and J. Sander, "Density-Based Clustering Based on Hierarchical Density Estimates," in *Advances in Knowledge Discovery and Data Mining*, vol. 7819, pp. 160–172, 2013.
- [49] F. Wang and H. Liu, "Understanding the Behaviour of Contrastive Loss," *arXiv*, 2021. [Online]. Available: <https://arxiv.org/abs/2012.09740>.
- [50] A. Kalisz, *et al.*, "B-SLAM-SIM: A Novel Approach to Evaluate the Fusion of Visual SLAM and GPS by Example of Direct Sparse Odometry and Blender," *VISIGRAPP*, 2019.
- [51] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3D Gaussian Splatting for Real-Time Radiance Field Rendering," *ACM Transactions on Graphics*, vol. 42, July 2023.
- [52] J. Xu, H. Le, V. Nguyen, V. Ranjan, and D. Samarasinghe, "Zero-Shot Object Counting," *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 15548–15557, 2023.